VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belagavi-560014

UNIX SYSTEM

PROGRAMMING

Mini Project Report

On

"POSIX-BASED SIMPLE TEXT EDITOR"

Submitted in partial fulfillment of the requirement of V Semester Unix System Programming with Mini Project (BCS515C)

Submitted by,

KHUSHBU KUMARI 1RF22IS040

Under the guidance of

Mrs. Swathi Darla

Asst. Professor

Dept. of ISE

RVITM, Bangalore.

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

RV INSTITUTE OF TECHNOLOGY AND MANAGEMENT®

Chaitanya Layout, JP Nagar 8th Phase, Kothanur, Bengaluru-560076

2023-2024

RV INSTITUTE OF TECHNOLOGY AND MANAGEMENT®

(Affiliated to Visvesvaraya Technological University, Belagavi & Approved by AICTE, New Delhi)

Chaitanya Layout, JP Nagar 8th Phase, Kothanur, Bengaluru-560076

DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING

Certificate

This is to certify that the mini-project work entitled "POSIX-BASED SIMPLE TEXT EDITOR" is carried out by **KHUSHBU KUMARI (1RF22IS040)** in partial fulfillment for the requirement of V Semester **Unix System Programming with Mini Project (BCS515C)** in **Information Science and Engineering** of the **Visvesvaraya Technological University, Belagavi** during the year 2023-2024. It is certified that all the corrections/ suggestions indicated for the given internal assessment have been incorporated in the report. This report has been approved as it satisfies the academic requirements with respect to the mini-project work.

Signature of the Guide	**Signature of the HOD**
Mrs. Swathi Darla	**Dr. Latha C A**
Asst. Professor, Dept. of ISE	Prof. & Head, Dept. of ISE
RVITM, Bangalore	RVITM, Bangalore.

ACKNOWLEDGEMENT

We express our profound gratitude to **Dr. Nagashettappa Biradar, Principal, RVITM, Bangalore,** for providing the necessary facilities and an ambient environment to work.

We are grateful to **Dr. Latha C A, Head of Department, Information Science and Engineering, RVITM, Bangalore,** for her valuable suggestions and advice throughout my/ourwork period.

We would like to thank our Project Guide, **Mrs. Swathi Darla** , Assistant Professor, Department of Information Science and Engineering, RV Institute of Technology and Management, Bengaluru, for her constant guidance and inputs.

We would like to thank all the staff members of **Department of Information Science and Engineering** for their support and encouragement during the course of this project.

ABSTRACT

This project presents a **POSIX-based Simple Text Editor** that utilizes the foundational principles of the **POSIX standard** for efficient and robust file management. Designed to operate in a Unix-like environment, the editor provides essential functionalities such as creating, reading, editing, and saving text files. The implementation demonstrates the use of both high-level scripting tools and low-level POSIX-compliant APIs, showcasing the versatility and power of the POSIX standard in real-world applications.

The editor is implemented in two levels. At the **shell scripting level**, it leverages command-line utilities like nano for editing and cat for reading, offering a menu-driven interface for intuitive user interaction. At the **application programming level**, it integrates key POSIX APIs, including open(), close(), read(), and write(), to manage files programmatically. This dual implementation provides users with both a practical tool and a hands-on opportunity to explore the underlying mechanisms of POSIX-based system programming.

The solution is developed on the **Ubuntu operating system**, chosen for its comprehensive support for POSIX-compliant features, stability, and user-friendly environment. Through this project, key Unix concepts such as file handling, process management, and system calls are explored, enhancing the understanding of POSIX programming and Unix utilities.

This project serves as a foundational tool for basic text manipulation while also acting as a learning platform for understanding POSIX-based file operations. By bridging the gap between shell scripting and system-level programming, it highlights the significance of POSIX in building efficient, reliable, and portable applications for Unix-based systems.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
1	ACKNOWLEDGEMENT	i
2	ABSTRACT	ii
3	TABLE OF CONTENTS	iii
4	INTRODUCTION	1-2
5	PROBLEM STATEMENT	3-5
6	METHODOLOGY	6-7
7	RESULTS AND SNAPSHOTS	8-13
8	CONCLUSION	14-16

CHAPTER 1

INTRODUCTION:

Overview of Unix System Programming (USP)

Unix System Programming (USP) is a specialized field of software development focused on programming and interacting with Unix-like operating systems. The system provides a variety of tools, shell utilities, and programming interfaces (APIs) that allow developers to interact directly with the underlying hardware and system services. The key aspects of USP include process control, file handling, memory management, system-level programming, and user management.

For a POSIX-based simple text editor, USP serves as the foundation for core operations like file creation, reading, editing, and saving. The project utilizes essential system calls such as open(), read(), write(), and close() to handle files. These calls allow precise control over file descriptors, enabling efficient and secure file management.

USP concepts such as standard input/output (STDIN_FILENO, STDOUT_FILENO) enable user interaction for menu-driven interfaces. Error handling through errno ensures robustness, while adherence to POSIX standards guarantees portability across Unix systems. Additionally, the integration of shell scripting with tools like nano and cat highlights the versatility of Unix utilities for basic file manipulation. This project demonstrates the power of USP in building lightweight, reliable applications while offering hands-on experience with system-level programming. By adhering to the POSIX standard, it ensures compatibility, making it a practical and educational tool for Unix-based environments.

Pre-requisites for the Project:

This project requires knowledge of:

- **POSIX File APIs**:
  - Understanding and using functions like open(), read(), write(), close(), and lseek() for precise control over file operations.
  - Exploring advanced features like file locking (fcntl()) to handle concurrent edits.

- **System-Level Programming**:
  - Learning system calls related to file management and user interaction.
  - Handling signals (e.g., SIGINT) to gracefully terminate the editor or save unsaved data when interrupted.
  - Managing processes to support multitasking, like opening multiple files simultaneously.

- **Shell Scripting**:
  - Automating repetitive file operations like creating backups, batch reading, or editing files using shell scripts.
  - Extending the editor's functionality with scripts to enhance user convenience.

- **API Development**:
  - Designing APIs for remote text file manipulation, such as creating, editing, or retrieving files stored on a server.
  - Utilizing RESTful APIs for integrating the editor with cloud storage or collaborative platforms.

- **Security**:
  - Implementing basic access control for file operations to prevent unauthorized edits.
  - Exploring JWT-based authentication if extending the editor to a remote or collaborative application.

Operating System and Version:

The project uses Ubuntu 20.04 LTS as the development environment due to its popularity in both development and production environments. Ubuntu is widely recognized for its:

- Stability: Known for long-term support (LTS), it is ideal for deployment and critical applications.
- Security: Ubuntu's built-in security mechanisms, such as AppArmor and firewall management, provide an added layer of protection for the system.
- Community Support: A massive community that offers troubleshooting assistance, detailed documentation, and frequent updates.
- Compatibility: Ubuntu is compatible with a wide range of hardware, making it versatile for various use cases.

Advantages of Ubuntu 20.04 LTS:

- Pre-configured Development Environment: Comes with all the necessary tools preinstalled for development, reducing the need for additional setup.

- Security: Regular patches and updates enhance system security, reducing vulnerabilities and providing a stable platform for mission-critical applications.

- Performance: Optimized to run efficiently even on lower-end hardware, ensuring better performance in production environments.

- Wide Adoption: Since Ubuntu is one of the most widely used Linux distributions, it is easy to find solutions to common problems and receive community support.

CHAPTER 2:

PROBLEM STATEMENT

Problem Identification:

The POSIX-based Simple Text Editor faces challenges in efficient file management, user interaction, and ensuring data integrity. Handling file creation, reading, writing, and error management using POSIX system calls can be complex. Providing a user-friendly command-line interface in a system-level environment without a GUI requires careful consideration. Concurrency issues like data corruption from simultaneous edits need to be addressed through file locking. Ensuring portability across different Unix-like systems while maintaining consistent functionality is crucial. Additionally, security concerns like unauthorized access and scalability for future features, such as remote editing, must be managed effectively.

Proposed Solution

The solution proposed in this project is a **POSIX-based Simple Text Editor** that integrates both shell scripting for local file management and system-level programming for efficient file operations. The editor allows users to create, read, edit, and save text files, leveraging POSIX-compliant functions for reliable and secure file handling.

1. **Shell Scripts**: Automate basic text file operations like creating, reading, and displaying file contents, providing a user-friendly command-line interface.
2. **System-Level Programming**: Implement core file management functionalities using POSIX system calls (open(), read(), write(), close()) for efficient handling of files and error management.

By combining these components, the editor provides both a simple, interactive shell-based interface and deeper access to the system's file handling capabilities. This approach not only ensures efficient file management but also guarantees portability and compatibility across different Unix-like systems.

Shell-Level vs API-Level Requirements:

- **Shell-Level Requirements**: The shell-level scripts manage essential file operations, including creating, opening, reading, editing, and saving text files. These scripts interact directly with POSIX system calls like open(), read(), write(), and close(), ensuring efficient file handling. The scripts also automate file permissions and error handling, leveraging Unix commands such as cat for displaying file content and nano for text editing. Additionally, shell utilities are used to manage file backups, handle file locking to prevent simultaneous edits, and manage user interactions through an intuitive command-line interface.

- **API-Level Requirements**: The API layer provides extended functionality for managing text files remotely. Built using Node.js and Express.js, this layer abstracts the shell-level file operations and allows users to perform file creation, editing, and viewing through HTTP requests. The API interacts with the underlying POSIX system calls to ensure seamless file manipulation. For secure access, JSON Web Tokens (JWT) are used for authentication, ensuring that only authorized users can edit or view files. This provides a secure and scalable approach to managing files in multi-user or networked environments.

Concepts Covered from UNIX:

- **File Management:**Commands and functions like open(), read(), write(), and close() are used for creating, reading, writing, and closing files, ensuring efficient file handling. These functions allow the editor to manage file content directly, while maintaining data integrity during read/write operations. File descriptors are managed appropriately to avoid resource leakage.

- **Text Editing Operations:**POSIX functions like lseek() and read() allow for precise text manipulation within the file, supporting features like line editing, file navigation, and content modification. These functions work together to facilitate reading from and writing to files efficiently, while also enabling error handling through errno.

- **Process and Signal Handling:**The editor may interact with signals to handle process interruptions and ensure data integrity. For example, the editor can implement signal handlers like SIGINT to handle unexpected interruptions (e.g., when the user presses Ctrl+C). These handlers ensure that any unsaved work is either saved or discarded properly.

- **File Permissions and Security:**Proper file permissions are set using chmod and chown to ensure that only authorized users can modify or access files. This is critical to prevent unauthorized changes to sensitive files, especially in multi-user environments. Additionally, secure access control methods can be implemented to restrict file access based on user roles.

Software and Hardware Requirements

Software Requirements:

- Operating System: Ubuntu 20.04 LTS or any Unix-like operating system with POSIX support.

- POSIX Libraries: Standard C libraries supporting timer and clock functions such as clock_gettime, timer_create, timer_settime, and clock_nanosleep.

- GCC or Compatible Compiler: For compiling C programs utilizing POSIX timer and clock APIs.

- Shell Scripting Environment: Bash or any POSIX-compliant shell for automating timer and clock-related operations.

- Optional API Tools: Node.js and Express.js if implementing an API layer for remote timer/clock management.

- Documentation Tools: Tools like Doxygen for generating documentation from code comments.

Hardware Requirements:

- Processor: Minimum dual-core processor for handling precise timer and clock operations and multitasking.
- Memory: 2GB RAM (minimum) or 4GB RAM (recommended) for smooth execution of programs and development tools.
- Disk Space: At least 5GB free space for storing project files, libraries, and logs.
- Real-Time Clock (RTC): Hardware with RTC support to ensure accurate timing, especially in embedded or real-time systems.
- Development System: Standard desktop or laptop for testing POSIX timer and clock functionality.

CHAPTER 03 METHODOLOGY

System Design:

The solution is built around a two-layer architecture: a local shell script layer for file operations and a remote API layer for managing files over HTTP.

1. **Shell Scripts** : The local shell scripts handle the core file operations, including creating, reading, editing, and saving text files. The scripts utilize POSIX system calls like open(), read(), write(), and close() to directly interact with the file system. Shell scripts automate tasks such as managing file permissions, creating backups, and ensuring efficient handling of text file operations in a secure and reliable manner.
2. **API Layer** : The API layer is built using Node.js and Express.js, providing a remote interface for managing files over HTTP. This API allows users to create, open, read, and save files through HTTP requests. It securely handles user access using JWT authentication, ensuring that only authorized users can modify or view files. The API interfaces with the shell scripts to perform the necessary file operations based on user input, making the system extensible and secure.

Detailed Code Explanation :

SHELL LEVEL :

```
#!/bin/bash

echo "Simple Text Editor - Shell-based"
while true; do
   echo "Menu:"
   echo "1. Create/Open a file"
   echo "2. Read a file"
   echo "3. Save and Exit"
   echo "Enter your choice:"
   read choice

   case $choice in
      1)
         echo "Enter the file name to create/open:"
         read file_name
         nano "$file_name"
         ;;
      2)
         echo "Enter the file name to read:"
         read file_name
         if [ -f "$file_name" ]; then
            cat "$file_name"
         else
```

```
 echo "File does not exist."
        fi
        ;;
    3)
        echo "Exiting the editor."
        break
        ;;
    *)
        echo "Invalid choice. Please try again."
        ;;
  esac
done
```

Explanation

- **Menu System**: Prompts the user to choose options like creating/opening files (nano), reading files (cat), or exiting.

- **Commands Used**:
  - nano: Opens or creates a file for editing.
  - cat: Displays file content.
  - chmod +x: Makes the script executable.

APPLICATION LEVEL :

```
 #include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>

#define BUFFER_SIZE 1024

void create_or_open_file() {
    char filename[100];
    printf("Enter the file name to create/open: ");
    scanf("%s", filename);

    int fd = open(filename, O_RDWR | O_CREAT, 0644);
    if (fd == -1) {
        perror("Error opening file");
        return;
```

```
    }

    printf("Enter content for the file (Ctrl+D to stop):\n");
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;

    while ((bytes_read = read(STDIN_FILENO, buffer, BUFFER_SIZE)) > 0) {
        if (write(fd, buffer, bytes_read) == -1) {
            perror("Error writing to file");
            close(fd);
            return;
        }
    }
    close(fd);
    printf("File saved successfully.\n");
}

void read_file() {
    char filename[100];
    printf("Enter the file name to read: ");
    scanf("%s", filename);

    int fd = open(filename, O_RDONLY);
    if (fd == -1) {
        perror("Error opening file");

   return;
    }

    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;
    printf("File content:\n");
    while ((bytes_read = read(fd, buffer, BUFFER_SIZE)) > 0) {
        if (write(STDOUT_FILENO, buffer, bytes_read) == -1) {
            perror("Error writing to stdout");
            close(fd);
            return;
        }
    }
    close(fd);
}
int main() {
    while (1) {
        printf("Menu:\n");
        printf("1. Create/Open a file\n");
```

```
        printf("2. Read a file\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");

        int choice;
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                create_or_open_file();
                break;
            case 2:
                read_file();
                break;
            case 3:
                printf("Exiting the editor.\n");
                exit(0);
            default:
                printf("Invalid choice. Please try again.\n");
        }
    }
    return 0;
}
```

Explanation

- **create_or_open_file()**: Uses open() with O_RDWR | O_CREAT flags to open or create a file. Reads input from the user and writes it to the file.
- **read_file()**: Opens a file in read-only mode (O_RDONLY) and displays its content using read() and write().

- **POSIX APIs Used**:
    - open(): Opens or creates a file.
    - close(): Closes the file descriptor.
    - read(): Reads content from a file or standard input.
    - write(): Writes content to a file or standard output.

CHAPTER 4:

SNAPSHOT AND RESULT

Shell level output:

```
Menu:
1. Create/Open a file
2. Read a file
3. Save and Exit
Enter your choice: 1
Enter the file name to create/open: testfile.txt
Menu:
1. Create/Open a file
2. Read a file
3. Save and Exit
Enter your choice: 2
Enter the file name to read: testfile.txt
This is the content of the file.
Menu:
1. Create/Open a file
2. Read a file
3. Save and Exit
Enter your choice: 3
Exiting the editor.
```

API Level Output:

```
Menu:
1. Create/Open a file
2. Read a file
3. Exit
Enter your choice: 1
Enter the file name to create/open: testfile.txt
Enter content for the file (Ctrl+D to save):
This is the content of the file.
File saved successfully.

Menu:
1. Create/Open a file
2. Read a file
3. Exit
Enter your choice: 2
Enter the file name to read: testfile.txt
File content:
This is the content of the file.

Menu:
1. Create/Open a file
2. Read a file
3. Exit
Enter your choice: 3
Exiting the editor.
```

Course Learning Outcomes:

This project covered the following learning outcomes:

- Enhancing knowledge of system-level I/O operations: Gained a deeper understanding of how input and output operations work at the system level using low-level functions like read() and write() for efficient data handling in files.
- Developing familiarity with file system management: Acquired insights into how the Unix file system operates, including concepts like file descriptors, file pointers, and file buffers, which are critical for handling file-based applications.
- Implementing user input validation and error management: Built robust mechanisms for validating user input and handling errors gracefully, ensuring that the program can manage invalid or incomplete inputs without crashing.
- Learning file locking mechanisms for safe multi-user editing: Explored basic file locking mechanisms in Unix to prevent concurrent write access to the same file, ensuring data integrity in multi-user environments.
- Gaining proficiency in using text editors through system calls: Integrated and interacted with existing text editors (e.g., nano or vim) via system calls to demonstrate how to create and modify files in a shell-based environment.
- Learning the significance of file descriptors: Improved understanding of file descriptors and how they are used to represent open files in memory, contributing to efficient file I/O operations.
- Understanding and handling different file types: Developed an understanding of handling various file types (regular files, directories, symbolic links) and their associated attributes using system calls.
- Understanding multi-threading and concurrent access to files: Gained knowledge of basic multi-threading concepts and how to implement file editing tools that could potentially handle multiple threads for concurrent editing operations in more advanced versions.
- Fostering debugging and problem-solving skills: Strengthened debugging skills by identifying and resolving issues related to memory management, file I/O operations, and system command interactions.
- Expanding knowledge of Unix utilities and commands: Increased familiarity with essential Unix commands like cat, ls, chmod, and chown, which were integrated to enhance the functionality of the text editor.
- Improving understanding of file system permissions and access control: Developed an understanding of advanced Unix file permissions and access control lists (ACLs) to ensure that files are accessed and modified securely.
- Mastering the basics of shell scripting for automation: Improved shell scripting skills by automating tasks related to text file creation, reading, and saving, making the process more efficient.
- Exploring debugging tools in Unix: Familiarized myself with debugging tools such as gdb and strace to monitor system calls and track issues in file I/O or process execution.
- Learning about process control and signals: Gained an understanding of how Unix processes work, including concepts of process control, signals like SIGTERM, SIGSEGV, and how to manage them for graceful program termination.
- Implementing basic file operations with user-friendly feedback: Focused on providing intuitive feedback to users for every action performed in the text editor, making it more user-friendly and responsive to errors.

Learning Experience:

Through this project, I gained a deeper understanding of:

- **Gained a deeper understanding of Unix-based file handling and system programming**: Developed a solid foundation in using POSIX system calls (open(), read(), write(), close()) to manipulate files in Unix environments.
- **Developed a strong grasp of file operations and permissions**: Learned how to implement secure file creation, reading, and writing operations, managing file permissions using commands like chmod and chown to ensure access control.
- **Enhanced skills in building command-line applications**: Strengthened my ability to design and implement user-friendly, interactive shell-based programs for text editing tasks, focusing on user input handling and menu-driven navigation.
- **Improved proficiency in error handling and debugging**: Gained hands-on experience in identifying and addressing common errors such as file access issues, read/write failures, and permission errors.
- **Acquired hands-on experience in system-level programming**: Applied theoretical knowledge in system-level programming, integrating POSIX system calls into a practical text editor application.
- **Strengthened knowledge of process management and signal handling**: Enhanced my understanding of how to handle process interruptions (e.g., SIGINT) and manage system signals in Unix-based environments to ensure program stability.
- **Applied theoretical concepts to build a practical, real-world solution**: Bridged the gap between theory and practice by building a functional and efficient text editor using low-level programming techniques.